

Practical: 7

Implementation of making a change problem using dynamic programming

/08/2026

Aim:

Implementation of making a change problem using dynamic programming

Algorithm:

Algorithm

Input : number of coins denominations n , coin array $\text{coins}[]$, and target amount A .

Output : minimum and maximum number of coins required to make amount A .

Step 1 : start

Step 2 : Read the number of coins n .

Step 3 : Read the n coins denominations into $\text{coins}[]$

Step 4 : Read the target amount A

Step 5 : create two DP arrays

- $\text{dpmin}[0 \dots A]$
- $\text{dpmax}[0 \dots A]$

Step 6 : initialize

$\text{dpmin}[0] = 0$
 $\text{dpmax}[0] = 0$

Step 7 : for every amount from 1 to A , check every coin $\text{coins}[i]$

Step 8 : for every amount from 1 to A , initialize

$\text{dpmin}[i] = \infty$
 $\text{dpmax}[i] = -\infty$

Step 9 : if $\text{coins}[i] \leq i$

Step 10 : calculate the minimum no of coins

$\text{dpmin}[i] = \min(\text{dpmin}[i], \text{dpmin}[i - \text{coins}[i]] + 1)$

calculate the maximum no of coins

$\text{dpmax}[i] = \max(\text{dpmax}[i], \text{dpmax}[i - \text{coins}[i]] + 1)$

Step 11 : Repeat Step 8-10 until all amounts from 1 to A are processed

Step 12 : Minimum no of coins is : $\text{dpmin}[A]$
Maximum no of coins is : $\text{dpmax}[A]$

Step 13 : stop

Program:

```
#include <iostream>
#include <algorithm>
using namespace std;

class CoinChange {
private:
    int coins[10];
    int n;
    int amount;

    int dpMin[100];
    int dpMax[100];

public:
    void getInput() {
        cout << "Enter number of coins: ";
        cin >> n;

        cout << "Enter coin denominations: ";
        for (int i = 0; i < n; i++) {
            cin >> coins[i];
        }

        cout << "Enter amount: ";
        cin >> amount;
    }

    void solve() {

        for (int i = 0; i <= amount; i++) {
            dpMin[i] = amount + 1;
            dpMax[i] = -1;
```

```
}

dpMin[0] = 0;
dpMax[0] = 0;

for (int i = 1; i <= amount; i++) {

    for (int j = 0; j < n; j++) {

        if (coins[j] <= i) {

            dpMin[i] = min(dpMin[i],
                           dpMin[i - coins[j]] + 1);

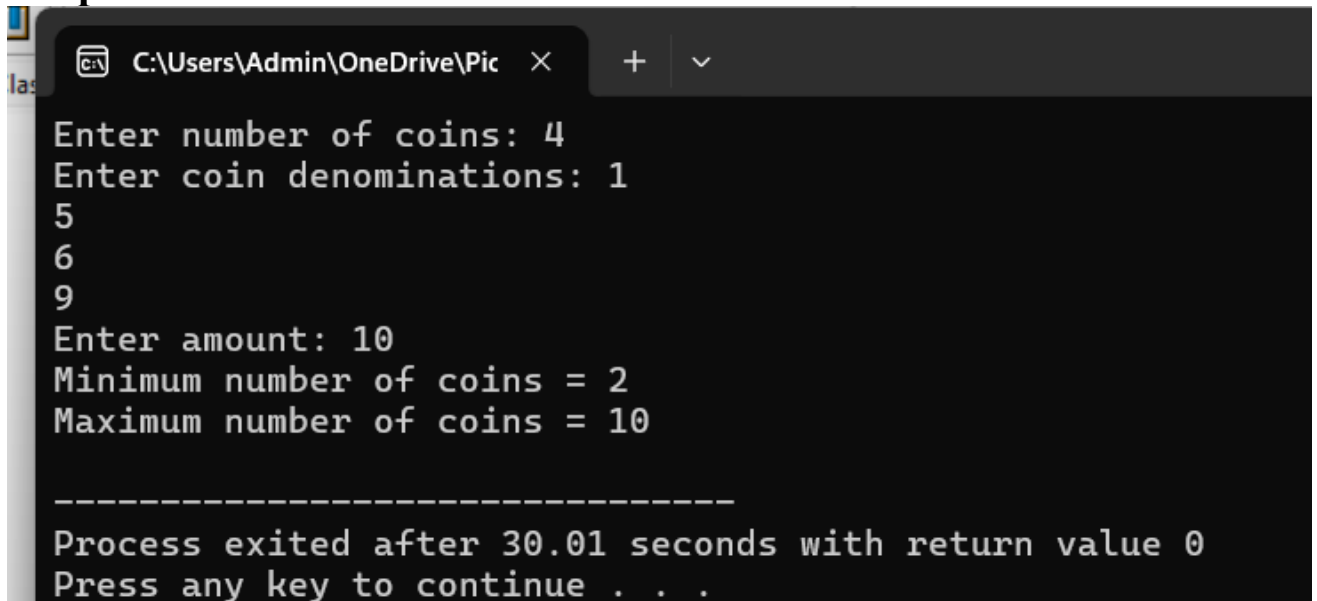
            if (dpMax[i - coins[j]] != -1) {
                dpMax[i] = max(dpMax[i],
                               dpMax[i - coins[j]] + 1);
            }
        }
    }
}

if (dpMin[amount] > amount) {
    cout << "Change cannot be made." << endl;
}
else {
    cout << "Minimum number of coins = "
         << dpMin[amount] << endl;

    cout << "Maximum number of coins = "
         << dpMax[amount] << endl;
}
}
```

```
};  
  
int main() {  
  
    CoinChange c;  
  
    c.getInput();  
    c.solve();  
  
    return 0;  
}
```

Output:



```
C:\Users\Admin\OneDrive\Pic X + v  
Enter number of coins: 4  
Enter coin denominations: 1  
5  
6  
9  
Enter amount: 10  
Minimum number of coins = 2  
Maximum number of coins = 10  
  
-----  
Process exited after 30.01 seconds with return value 0  
Press any key to continue . . .
```

Time Complexity:

Time complexity

let n = number of coin denominations
 A = target amount

Solve function

The program contain two nested loops

```

for (inti=1; i<=amount; i++) {
    for (intj=0; j<n; j++) {

```

- The outer loop executes A times
- The inner loop ~~at~~ every amount, all n contain coins are checked

Therefore, the nested loops take

$T(n, A) = O(A) \times O(n)$
 $O(nA)$

Time complexity : $T(n, A) = O(nA)$

Best case : even if a solution is found immediately, the loops continue

$T(n) = O(nA)$

Average case : on a average, the same dp table is filled and all coins are considered

$T(n) = O(nA)$

Worst case : All amounts and all coins denominations are processed

$T(n) = O(nA)$

Conclusion:

The Making Change Problem was successfully implemented using Dynamic Programming to determine both the minimum and maximum number of coins required to make a given amount. The method efficiently solves the problem by storing solutions to smaller subproblems and avoids repeated calculations.